

Quy trình Render 3D Gaussian Splatting và Cải tiến Structure-Aware (SADGS)

45 slide — đối chiếu trực tiếp với code thật trong repo

Digital Twin GS

Ngày 27 tháng 9 năm 2026

Vì sao 3D Gaussian Splatting?

- **Bài toán đặt ra:** cho một tập ảnh chụp một cảnh từ nhiều góc, cùng camera pose ước lượng sẵn bởi SfM/COLMAP, hãy dựng lại một biểu diễn 3D sao cho có thể **render lại ảnh từ bất kỳ góc nhìn mới nào** (novel view synthesis) một cách chân thực.
- **NeRF (2020)** giải bài này bằng một mạng MLP ẩn ánh xạ (vị trí, hướng nhìn) $\rightarrow$ (màu, mật độ), rồi **ray-marching**: bắn hàng trăm điểm mẫu dọc mỗi tia sáng, gọi mạng hàng trăm lần cho mỗi pixel. Chất lượng cao nhưng cực chậm — train mất hàng giờ/ngày, render cũng mất giây/khung hình vì phải lặp lại forward pass qua mạng cho từng điểm mẫu.
- **3D Gaussian Splatting** (Kerbl, Kopanas, Leimkühler, Drettakis — SIGGRAPH 2023) đổi hẳn cách biểu diễn: thay mạng ẩn bằng **hàng trăm nghìn tới hàng triệu “Gaussian 3D” tường minh**, mỗi Gaussian là một đối tượng có tham số cụ thể $\{\mu, \Sigma, \alpha, c\}$ (Slide sau). Render không cần mạng nơ-ron, không cần ray-marching — chỉ cần **chiếu (project) và tô (rasterize)** trực tiếp các Gaussian lên màn hình, giống hệt cách GPU vẽ tam giác trong đồ họa game truyền thống. Kết quả: train nhanh hơn nhiều lần, và quan trọng nhất — render **thời gian thực** (hàng chục-hàng trăm khung hình/giây) sau khi train xong.
- **Nhưng còn một vấn đề chưa giải quyết trọn vẹn:** số lượng Gaussian N cần bao nhiêu, đặt ở đâu, kích thước ra sao — không biết trước, phải **tự động điều chỉnh trong lúc train** (đây gọi là *Adaptive Density Control*, viết tắt ADC). Cơ chế ADC gốc chỉ dựa vào **gradient** để quyết định thêm/bớt Gaussian — và đây chính là chỗ **SADGS** (Structure-Aware Densification for Gaussian Splatting) can thiệp: bổ sung thêm một tín hiệu đo trực tiếp **cấu trúc tần số cục bộ** của ảnh, để quyết định “đúng chỗ, đúng lúc, đúng trục” hơn.

Lộ trình bài giảng: (1) Biểu diễn Gaussian & pipeline render $\rightarrow$ (2) Loss, gradient, ADC nguyên bản của 3DGS $\rightarrow$ (3) SADGS: xây dựng tín hiệu tần số η từ đầu, cơ chế densify/prune mới $\rightarrow$ (4) Nhìn lại một cách trung thực: cái gì đang thật sự chạy, so sánh với bản gốc.

3D Gaussian: đơn vị biểu diễn cơ bản

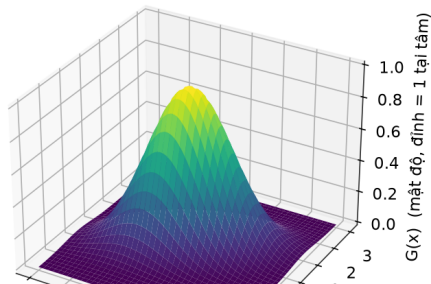
Toàn bộ cảnh được biểu diễn bằng một **tập hợp các Gaussian 3D** — mỗi Gaussian là một hàm mật độ liên tục trong không gian, tâm tại $\mu \in \mathbb{R}^3$, "độ loang" theo ma trận hiệp phương sai $\Sigma \in \mathbb{R}^{3 \times 3}$:

$$G(x) = \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right)$$

Có thể hình dung mỗi Gaussian như một **"cục mây mờ" hình ellipsoid**: đậm đặc nhất ở tâm μ , mờ dần ra biên theo hình dạng do Σ quyết định. Mỗi Gaussian mang theo 4 nhóm tham số **học được bằng gradient descent** (tổng 59 số thực: xyz 3 + scaling 3 + rotation 4 + opacity 1 + f_dc 3 + f_rest 45):

- μ — vị trí tâm trong không gian 3D (3 số).
- Σ — hình dạng và hướng của ellipsoid, không lưu trực tiếp mà tham số hoá qua scale $s \in \mathbb{R}^3$ và quaternion (hướng quay) $q \in \mathbb{R}^4$ — lý do tại sao xem ở slide tiếp theo.
- $\alpha \in (0, 1)$ — độ mờ đục (opacity): Gaussian "đặc" hay "trong suốt", lưu dưới dạng logit rồi đi qua sigmoid để luôn nằm trong khoảng hợp lệ.
- c — màu sắc, nhưng **không phải một màu RGB cố định** mà biểu diễn bằng hệ số Spherical Harmonics (Slide sau) để màu có thể *đổi theo góc nhìn* — mô phỏng hiệu ứng phản chiếu, ánh kim, specular thực tế.

$$G_i(x) = \exp\left(-\frac{1}{2}(x - \mu_i)^\top \Sigma_i^{-1}(x - \mu_i)\right)$$



Tham số hoá hiệp phương sai: luôn xác định dương

Về mặt toán học, Σ phải **đối xứng và xác định dương** mới là một ma trận hiệp phương sai hợp lệ (nếu không, $G(x)$ ở trên có thể phát tán vô nghĩa). Nếu tối ưu trực tiếp 6 số độc lập của Σ bằng gradient descent, rất dễ trong quá trình huấn luyện đi lệch khỏi vùng hợp lệ. Giải pháp của 3DGS là **phân rã** Σ thành tích của một ma trận quay và một ma trận scale:

$$\Sigma = RSS^T R^T, \quad S = \text{diag}(s_x, s_y, s_z), \quad R = R\left(\frac{q}{\|q\|}\right)$$

- S : ma trận đường chéo với $s = \exp(\texttt{_scaling})$ — vì hàm mũ luôn dương, s **tự động dương** với mọi giá trị thực của $\texttt{_scaling}$, không cần ràng buộc (constraint) nào thêm khi tối ưu.
- R : ma trận quay 3×3 dựng từ quaternion q đã **chuẩn hoá về độ dài 1** — luôn là ma trận trực giao hợp lệ.
- Nhờ vậy $\Sigma = RSS^T R^T$ **tự động** đối xứng, xác định dương với **bất kỳ** s, q nào — ta có thể tối ưu s, q tự do bằng gradient descent mà không sợ vi phạm ràng buộc toán học.
- Về mặt hình học: ellipsoid của Gaussian có **bán trục theo mỗi hướng chính** $= s_j$ (không phải trị riêng của Σ — trị riêng thực ra là s_j^2 , vì Σ là bình phương của S sau khi quay).

Khởi tạo (1/2): scale ban đầu theo mật độ điểm SfM

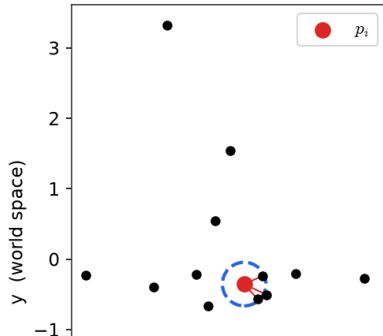
Trước khi train, mỗi Gaussian được đặt **đúng tại vị trí** một điểm 3D thưa mà SfM/COLMAP đã tái dựng được từ các ảnh input (*sparse point cloud*). Nhưng vị trí thôi chưa đủ — cần một **kích thước ban đầu hợp lý** cho mỗi Gaussian: quá nhỏ thì để hờ nhiều khoảng trống giữa các điểm, quá to thì các Gaussian chồng lấn nhau ngay từ đầu, làm nhiễu tín hiệu học. Giải pháp: lấy khoảng cách tới các điểm lân cận làm thước đo mật độ cục bộ:

$$\tilde{s} = \log \sqrt{d_{\text{knn}3}^2}$$

tức là: với mỗi điểm, tính khoảng cách bình phương tới 3 điểm gần nhất (knn3), lấy trung bình, rồi căn bậc hai — ra một ước lượng “khoảng cách điển hình giữa các điểm lân cận” tại vị trí đó. Lấy log vì scale được lưu và tối ưu trong không gian log (nhất quán với $s = \exp(\text{_scaling})$ ở slide trước). Kết quả: vùng point cloud dày đặc (nhiều chi tiết hình học) sẽ có Gaussian khởi tạo **nhỏ**, vùng thưa sẽ có Gaussian khởi tạo **to** hơn — một cách tự nhiên để phủ kín cảnh mà không cần biết trước hình dạng vật thể.

$$\tilde{s}_i = \log \sqrt{d_{\text{knn}3}^2(p_i)} \cdot \vec{1}_3$$

(bán kính = RMS khoảng cách tới 3 láng giềng)

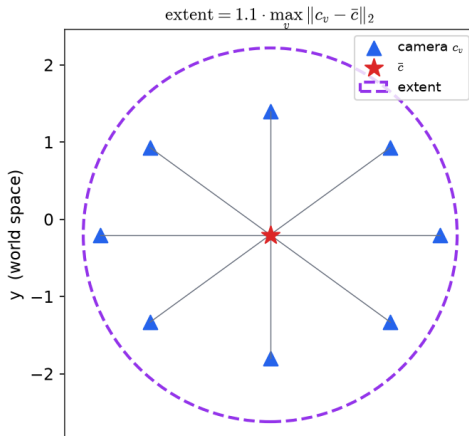


Khởi tạo (2/2): scene extent — thước đo "to/nhỏ" dùng xuyên suốt

Nhiều quyết định sau này trong ADC (Adaptive Density Control) cần biết "Gaussian này to hay nhỏ **so với quy mô của cả cảnh**" — ví dụ ngưỡng $\text{percent_dense} \times \text{extent}$ để phân biệt clone/split (sẽ nói ở phần sau). Đại lượng dùng làm đơn vị đo lường này là **scene extent**, tính **một lần duy nhất** lúc khởi tạo:

$$\text{extent} = 1.1 \cdot \max_v \|c_v - \bar{c}\|$$

với c_v là vị trí của camera train thứ v , $\bar{c}$ là tâm trung bình của tất cả camera train. Nói cách khác, đây là **bán kính** (nhân thêm hệ số an toàn 1.1) của một khối cầu tưởng tượng bao quanh toàn bộ vị trí các camera — **không phải** đường chéo của hộp bao (bounding box) quanh point cloud như đôi khi bị hiểu nhầm. Vì cảnh thường được quay quanh một đối tượng trung tâm, "bán kính vòng camera" là một ước lượng hợp lý cho kích thước tổng thể của cảnh.



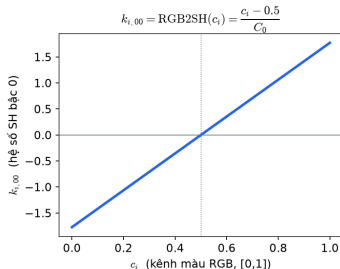
Màu phụ thuộc góc nhìn: hệ số Spherical Harmonics

Vật liệu thực tế (kim loại, nước, sứ bóng...) phản chiếu ánh sáng khác nhau tùy góc nhìn — một màu RGB cố định không mô phỏng được hiệu ứng này. 3DGS giải quyết bằng cách biểu diễn màu như một **hàm của hướng nhìn** d , khai triển theo cơ sở **Spherical Harmonics** (SH) — một họ hàm trực giao trên mặt cầu, tương tự chuỗi Fourier nhưng cho dữ liệu định nghĩa trên hình cầu thay vì trên một đường thẳng:

$$c(d) = \sum_{\ell=0}^D \sum_{m=-\ell}^{\ell} k_{\ell m} Y_{\ell}^m(d)$$

mỗi Gaussian lưu các **hệ số** $k_{\ell m}$ (học được), không lưu trực tiếp màu.

- Hệ số bậc 0 ($\ell = 0$, gọi là **hệ số DC**) chính là **màu trung bình theo mọi góc nhìn**: khởi tạo $k_{00} = \text{RGB2SH}(c) = (c - 0.5)/C_0$ với hằng số chuẩn hoá $C_0 = \frac{1}{2\sqrt{\pi}} \approx 0.282$.
- Các hệ số bậc cao hơn ($\ell \geq 1$) mã hoá **độ thay đổi màu theo hướng nhìn** — càng nhiều bậc, hiệu ứng phản chiếu/specular càng chi tiết, nhưng cũng càng nhiều tham số phải học.
- Tổng số hệ số cần học nếu dùng tới bậc D : $(D + 1)^2$ hệ số cho mỗi kênh màu; với $D = 3$ (giá trị dùng phổ biến) là 16 hệ số/kênh $\times 3$ kênh = 48 số (3 số DC + 45 số bậc cao, tên biến `f_rest`).

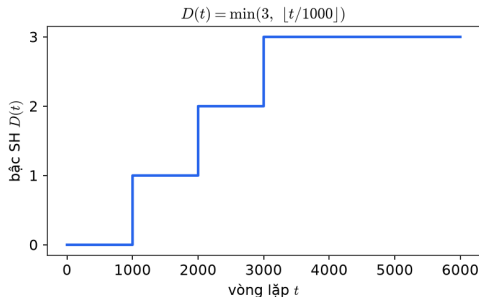


Lịch tăng bậc SH: vì sao không học bậc cao ngay từ đầu?

Học ngay 48 hệ số SH từ những iteration đầu tiên, khi vị trí/hình dạng Gaussian còn rất chưa ổn định, dễ làm mô hình “khớp” (overfit) vào những biến thiên màu do **sai vị trí hình học** gây ra, chứ không phải phản chiếu thật — gây nhiễu và bất ổn cho quá trình tối ưu. Giải pháp: **tăng bậc SH dần dần**, để hình học ổn định trước, chi tiết phản chiếu học sau. Trong code (train.py:216--217):

```
if iteration % 1 == 0: gaussians.oneupSHdegree()
```

tức bậc **tăng thêm 1 sau mỗi iteration**, cho tới khi chạm bậc tối đa $D_{\max} = 3$ — nghĩa là chỉ sau **3 iteration đầu tiên** của cả quá trình train (30 000 iteration), toàn bộ Gaussian đã dùng bậc SH tối đa. Đây là một điểm dễ hiểu lầm: nhiều tài liệu khác về 3DGS mô tả lịch tăng bậc là “mỗi vài nghìn iteration tăng 1 bậc” — con số đó **không khớp** với đoạn code thực tế của chính pipeline này.



Chiếu điểm 3D lên màn hình: mô hình pinhole

Để vẽ một Gaussian 3D lên ảnh 2D, bước đầu tiên là biết **tâm của nó rơi vào pixel nào**. Camera pinhole (mô hình camera tiêu chuẩn trong thị giác máy tính) chiếu một điểm (x, y, z) trong hệ toạ độ camera thành pixel (u, v) theo phép chiếu phối cảnh cổ điển:

$$u = f_x \frac{x}{z} + c_x, \quad v = f_y \frac{y}{z} + c_y$$

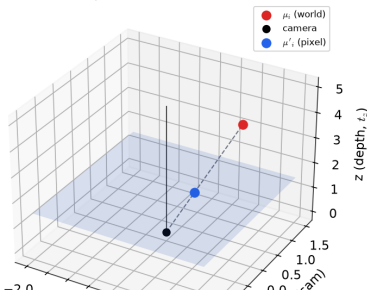
với (f_x, f_y) là tiêu cự tính theo đơn vị pixel (phụ thuộc độ phân giải ảnh và góc nhìn của camera), (c_x, c_y) là **điểm chính** (thường là chính giữa ảnh, $W/2, H/2$).

- Đây là phép chiếu **phi tuyến** vì có phép chia cho z (độ sâu) — vật càng xa càng nhỏ lại, đúng như cảm nhận thị giác thông thường.
- Vấn đề: Gaussian không phải một điểm mà là cả một **ellipsoid 3D** — phép chiếu phi tuyến của cả một khối hình không cho ra một ellipse đẹp một cách chính xác. Giải pháp thực dụng: **tuyến tính hoá cục bộ** quanh tâm Gaussian bằng ma trận Jacobian J (đạo hàm riêng của phép chiếu tại đúng điểm đó):

$$J = \begin{pmatrix} f_x/z & 0 & -f_x x/z^2 \\ 0 & f_y/z & -f_y y/z^2 \end{pmatrix}$$

Ma trận J này sẽ được dùng lại ở **cả hai nơi**: chiếu hiệp phương sai sang 2D ngay sau đây, và chiếu trục Gaussian của cơ chế SADGS về sau.

$$p^{proj} = p_{xyz}^{hom} / p_w^{hom}, \quad \mu'_i = \text{pixel coords}$$



EWA splatting: chiếu ellipsoid 3D thành ellipse 2D

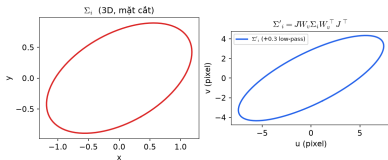
Dùng Jacobian J ở slide trước, ta xấp xỉ tuyến tính phép chiếu của toàn bộ hiệp phương sai Σ (kỹ thuật *Elliptical Weighted Average*, Zwicker et al. 2001 — nền tảng toán học cho mọi hệ splatting hiện đại):

$$\Sigma' = [JW\Sigma W^T J^T]_{2 \times 2} + \kappa I$$

W là ma trận quay world $\rightarrow$ camera; $[\cdot]_{2 \times 2}$ nghĩa là chỉ giữ lại khối 2×2 trên-trái của kết quả (bỏ chiều depth, vì ta chỉ cần ellipse trên mặt phẳng ảnh).

- **Vì sao cần $+\kappa I$?** Khi Gaussian ở rất xa camera hoặc bị nhìn gần như từ cạnh, ellipse chiếu ra có thể suy biến thành một đường gần như không có bề rộng — gây chia cho 0 và alias (răng cưa). Cộng thêm κI đảm bảo ellipse luôn rộng **ít nhất khoảng 1 pixel** theo mọi hướng — một bộ lọc low-pass đơn giản, **luôn được áp dụng**, không phải tùy chọn.
- Trong CUDA kernel thực tế của repo này (`cuda_rasterizer/forward.cu:113--117`), $\kappa = 0.1$ — khác với con số 0.3 vẫn thường thấy trong một số tài liệu 3DGS khác.
- Vì “phồng” hiệp phương sai làm sai lệch tổng năng lượng của Gaussian, renderer nhân bù ngược một **hệ số hiệu chỉnh**
 $\text{coef} = \sqrt{\det \Sigma'_{\text{trước}} / \det \Sigma'_{\text{sau}}}$ vào α để giữ năng lượng gần đúng.

Chiếu covariance: 3D ellipsoid $\rightarrow$ 2D ellipse trên ảnh



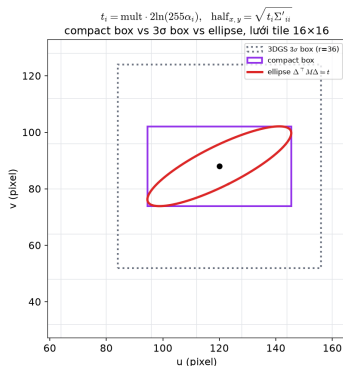
Compact box: xác định Gaussian phủ những tile nào

Sau khi có ellipse 2D, renderer cần biết nó **chạm tới những tile** 16×16 **pixel** nào trên màn hình — đây là bước quyết định renderer nhanh hay chậm (Gaussian chạm càng nhiều tile, càng tốn công xử lý). Repo này dùng kỹ thuật **SnugBox** (`cuda_rasterizer/auxiliary.h:313--340`, kỹ thuật kế thừa từ dòng nghiên cứu Taming/Speedy-Splat, **không phải điểm mới của SADGS**) — thay vì bao Gaussian bằng một hình vuông cố định 3σ như 3DGS gốc, SnugBox dựng một hộp bao **khít sát theo đúng hình dạng ellipse thật**, tại ngưỡng nơi $\alpha \cdot G = 1/255$ (dưới mức này coi như không nhìn thấy được nữa), có thể siết chặt hơn nữa bằng một hệ số điều chỉnh `mult`:

$$t = \text{mult} \cdot 2 \ln(255 \alpha)$$

(`mult = 0.7` mặc định, `arguments/__init__.py:114`)

- `mult = 1`: khớp đúng ngưỡng $1/255$ gốc (SnugBox nguyên bản, không cắt xén thêm gì).
- `mult < 1`: cắt **sớm hơn** $1/255$ — ngưỡng cắt thật trở thành $\alpha_{\text{biên}} = \sigma_o(255\sigma_o)^{-\text{mult}} > 1/255$, nghĩa là đánh đổi một phần đuôi Gaussian *vẫn còn nhìn thấy được* để lấy tốc độ (ít tile hơn/Gaussian $\Rightarrow$ ít công tính hơn).

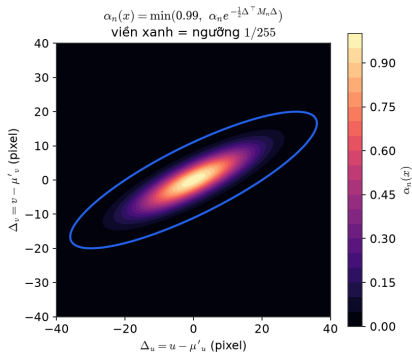


Trường alpha: mỗi Gaussian che phủ pixel bao nhiêu?

Một Gaussian không "đặc" đều — độ che phủ giảm dần từ tâm ra biên theo đúng hàm $G(x)$. Tại một pixel cụ thể x trên màn hình, độ mờ đục hiệu dụng của Gaussian thứ n là:

$$\alpha_n(x) = \min(0.99, \alpha_n G_n(x))$$

- α_n (tham số học được, cố định cho cả Gaussian) nhân với $G_n(x)$ (giá trị hàm Gaussian 2D đã chiếu, **thay đổi theo từng pixel** x — cao nhất tại tâm ellipse, giảm dần ra ngoài).
- Chặn trên 0.99 (không phải 1.0): tránh một Gaussian duy nhất "che kín hoàn toàn" khiến gradient của mọi Gaussian phía sau nó biến mất hẳn — luôn để lại một khe hở rất nhỏ cho tín hiệu gradient lan truyền.
- Đây chính là đại lượng $\alpha_n(x)$ sẽ được dùng trong công thức alpha blending ở slide tiếp theo — lưu ý nó phụ thuộc pixel, khác với α_n hằng số của riêng Gaussian.

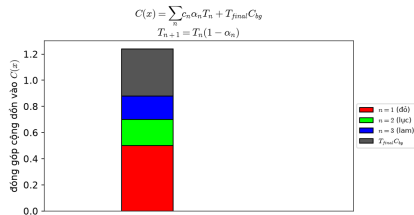


Alpha blending: trộn nhiều lớp Gaussian theo thứ tự trước-sau

Tại một pixel, có thể có **nhều Gaussian** chồng lên nhau theo chiều sâu. Màu cuối cùng là kết quả trộn tuần tự từ Gaussian gần camera nhất ra xa nhất (front-to-back alpha blending, giống cách vẽ các lớp trong suốt chồng lên nhau):

$$C(x) = \sum_n c_n \alpha_n(x) T_n(x) + T_{\text{final}}(x) C_{bg}, \quad T_n(x) = \prod_{j < n} (1 - \alpha_j(x))$$

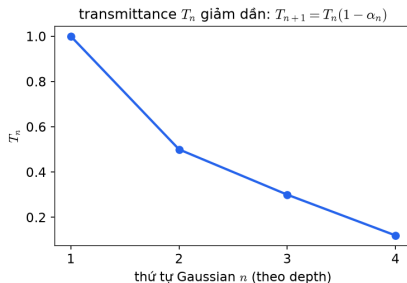
- T_n (**transmittance**): tỉ lệ ánh sáng *còn lại*, chưa bị các Gaussian phía trước "hấp thụ" — Gaussian thứ n chỉ đóng góp đúng phần ánh sáng nó nhận được sau khi đã đi qua $n - 1$ lớp trước nó.
- Số hạng $T_{\text{final}} C_{bg}$ (màu **nền**) là bắt buộc, không được bỏ qua: nếu tổng $\alpha_n(x)$ dọc một tia không phủ hết 1 (ví dụ vùng thưa Gaussian), phần còn thiếu phải được lấp bằng màu nền, nếu không ảnh sẽ bị tối sai so với thực tế.



Transmittance suy giảm: cơ sở cho việc dừng sớm

Vì $T_n = \prod_{j < n} (1 - \alpha_j(x))$ là tích của các thừa số ≤ 1 , T_n **giảm dần theo cấp số nhân** khi đi qua ngày càng nhiều lớp Gaussian (đặc biệt rõ khi từng $\alpha_j(x)$ đều đáng kể). Sau một số lớp đủ nhiều, T_n trở nên cực nhỏ — các Gaussian tiếp theo dù có màu gì cũng gần như **không còn ảnh hưởng** tới $C(x)$ nữa vì bị nhân với một hệ số gần 0.

- Renderer tận dụng điều này để **dừng sớm** (early termination): ngay khi $T \cdot (1 - \alpha) < 10^{-4}$ tại một pixel, ngừng xử lý thêm Gaussian cho pixel đó — tiết kiệm đáng kể thời gian tính toán mà hầu như không ảnh hưởng chất lượng ảnh (sai số dưới ngưỡng nhận biết được).
- Đây cũng là lý do vì sao **thứ tự depth** (Gaussian nào gần camera hơn) rất quan trọng: nếu sắp sai thứ tự, dừng sớm có thể bỏ lỡ một Gaussian đáng lẽ vẫn còn đóng góp đáng kể.



Kiến trúc rasterizer theo tile: vì sao render nhanh

Với hàng triệu Gaussian và hàng triệu pixel, việc lặp “cho mỗi pixel, quét qua mọi Gaussian” là bất khả thi về tốc độ. 3DGS giải quyết bằng kiến trúc **song song hoá theo tile**, chia màn hình thành các ô 16×16 pixel:

- ➊ **Xác định phạm vi:** với mỗi Gaussian, dùng compact box (slide trước) để biết nó phủ tới những tile nào $\Rightarrow$ sinh ra các cặp khoá (chỉ số tile, độ sâu) — một Gaussian phủ nhiều tile thì sinh nhiều cặp khoá, mỗi cặp ứng với một tile.
- ➋ **Sắp xếp một lần duy nhất:** toàn bộ danh sách cặp khoá được sắp theo (tile, depth) — đây là bước tốn kém nhất về mặt lý thuyết, nhưng chỉ thực hiện **một lần cho cả khung hình**, không phải một lần riêng cho từng pixel như cách làm ngây thơ.
- ➌ **Xử lý song song theo tile:** mỗi *thread block* trên GPU phụ trách đúng một tile; vì danh sách đã sắp sẵn theo depth, mỗi thread chỉ cần blend tuần tự (công thức alpha blending, slide trước) theo đúng thứ tự, dừng sớm khi T đủ nhỏ.

Kết quả: tổng chi phí gần với $O(N \cdot K)$, với K là số tile trung bình mà mỗi Gaussian chạm tới (bị chi phối trực tiếp bởi *mult* của compact box) — thay vì $O(N \cdot H \cdot W)$ nếu phải quét toàn ảnh cho mỗi Gaussian. Đây là lý do cốt lõi giúp 3DGS render **thời gian thực**.

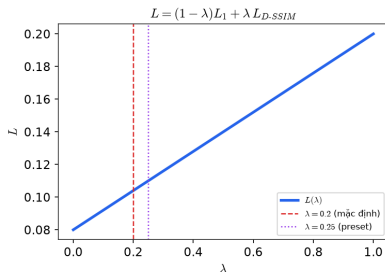
Hàm mất mát: so khớp ảnh render với ảnh thật

Sau khi render ra ảnh I_{render} , cần một hàm đo "khác biệt" với ảnh thật I_{gt} để lấy gradient huấn luyện. Trong repo này (`train.py:256--259`):

$$\mathcal{L} = (1 - \lambda_{\text{dssim}}) L_1 + \lambda_{\text{dssim}} (1 - \text{SSIM}) + \lambda_{L_2} L_2$$

với $\lambda_{\text{dssim}} = 0.2$, $\lambda_{L_2} = 2.0$ — so với 3DGS gốc (chỉ có hai số hạng đầu), bản này **cộng thêm** một số hạng L_2 (bình phương sai số theo pixel, `utils/loss_utils.py:25`).

- $L_1 = |I_{\text{render}} - I_{\text{gt}}|$: ổn định, ít bị kéo lệch bởi vài pixel ngoại lệ (outlier) — một vài pixel sai rất nhiều (do noise, occlusion...) không làm gradient tổng thể "hoảng loạn".
- SSIM (Structural Similarity, cửa sổ Gaussian 11×11 , $\sigma = 1.5$): so khớp **cấu trúc cục bộ** — độ tương phản, độ chói theo từng vùng nhỏ — gần với cảm nhận thị giác con người hơn so sánh từng pixel riêng lẻ.
- L_2 (bình phương): phạt **nặng hơn** với sai số lớn, giúp mô hình hội tụ nhanh ở giai đoạn đầu khi ảnh render còn rất khác ảnh thật.
- **Ghi chú quan trọng:** `loss_utils.py` còn định nghĩa hai hàm `tone_curve_loss` và `frequency_loss` (dựa trên structure tensor sẽ nói ở phần SADGS) — nhưng **không có dòng nào trong `train.py` gọi tới hai hàm này**. Đây là *dead code*: tồn tại trong file nhưng không ảnh hưởng gì tới loss thật sự đang được tối ưu.

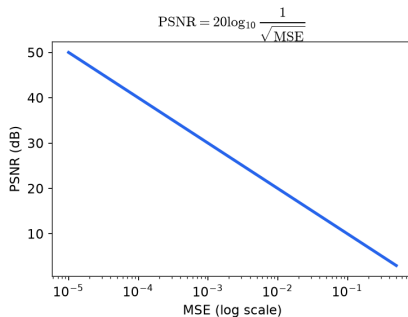


Đánh giá chất lượng (1/3): PSNR

Sau khi train xong, cần các chỉ số **độc lập với loss** để đánh giá khách quan chất lượng render trên tập test (ảnh chưa dùng để train). Chỉ số đầu tiên và đơn giản nhất là **PSNR** (Peak Signal-to-Noise Ratio):

$$\text{PSNR} = 10 \log_{10} \left(\frac{1}{\text{MSE}} \right) \quad (\text{dB, với ảnh chuẩn hoá về } [0, 1])$$

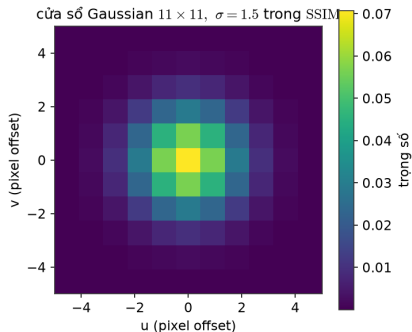
- PSNR càng **cao** nghĩa là sai số bình phương trung bình theo pixel càng **thấp** — là thang đo logarit nên chênh lệch vài dB đã tương ứng với chênh lệch sai số đáng kể.
- **Hạn chế:** PSNR đối xử với mọi pixel như nhau, không phân biệt "sai ở vùng chi tiết phức tạp" hay "sai ở vùng phẳng đơn giản" — hai ảnh có cùng PSNR có thể trông rất khác nhau về mặt cảm nhận thị giác (một ảnh mờ đều, một ảnh có vài vết lỗi rõ rệt nhưng phần còn lại rất sắc nét).
- Vì hạn chế này, PSNR thường được báo cáo **cùng với** SSIM và LPIPS (hai slide tiếp theo), không dùng một mình.



Đánh giá chất lượng (2/3): SSIM

SSIM (Structural Similarity Index) — cùng công thức đã dùng làm một phần của loss (slide trước) — nhưng ở đây dùng để **đánh giá**, không phải để lấy gradient. So sánh ảnh theo **cửa sổ trượt cục bộ** (Gaussian 11×11 , $\sigma = 1.5$), kết hợp ba thành phần: độ chói (luminance), độ tương phản (contrast), và cấu trúc (structure) giữa hai ảnh trong mỗi cửa sổ.

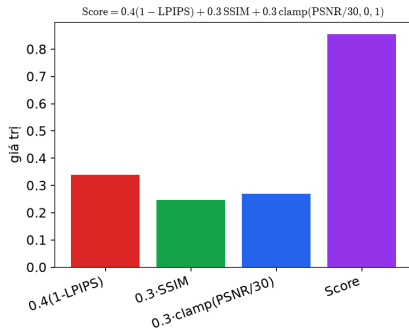
- Giá trị nằm trong $[-1, 1]$ (thường báo cáo trong $[0, 1]$ với ảnh tự nhiên), 1 là khớp hoàn hảo.
- Vì so sánh theo **vùng cục bộ** thay vì từng pixel riêng lẻ, SSIM nhạy với sai lệch về **cấu trúc** (đường nét, hoạ tiết bị mờ/lệch) hơn là sai lệch nhỏ về độ sáng tổng thể — gần với cách mắt người đánh giá “ảnh này có giống ảnh kia không” hơn PSNR thuần.
- Trong 3DGS, SSIM đóng vai trò **kép**: vừa là một phần của hàm loss lúc train (khuyến khích khớp cấu trúc), vừa là chỉ số đánh giá lúc test.



Đánh giá chất lượng (3/3): kết hợp nhiều chỉ số

Ba chỉ số PSNR, SSIM, và **LPIPS** (Learned Perceptual Image Patch Similarity — khoảng cách giữa hai ảnh đo trong không gian đặc trưng của một mạng CNN đã huấn luyện trước trên dữ liệu thị giác con người) đo những khía cạnh **khác nhau và bổ sung cho nhau**:

- PSNR: sai số pixel thô, dễ tính, dễ hiểu, nhưng ít tương quan với cảm nhận thị giác.
- SSIM: cấu trúc cục bộ, tương quan tốt hơn với cảm nhận, nhưng vẫn là một công thức toán học tường minh, không "học" từ dữ liệu thị giác người.
- LPIPS: học trực tiếp từ dữ liệu đánh giá của con người, bắt được các khác biệt "nhìn thấy rõ" mà PSNR/SSIM có thể bỏ sót (ví dụ: một chi tiết bị làm mờ nhẹ nhưng vẫn giữ đúng cấu trúc trung bình).
- Không chỉ số nào là "đủ một mình" — một báo cáo kết quả nghiêm túc nên nêu **cả ba**, kèm rõ scene, checkpoint (số iteration), và cấu hình đo, để tránh so sánh khập khiễng giữa các thiết lập khác nhau.

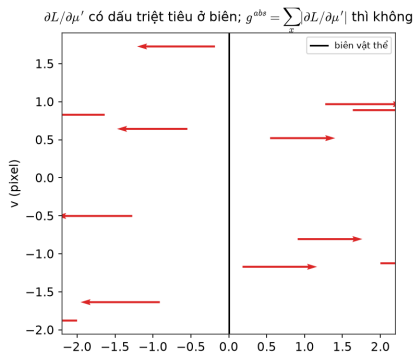


Lan truyền gradient: từ loss ngược về từng tham số Gaussian

Toàn bộ pipeline vừa trình bày — chiếu, EWA, alpha blending — là một chuỗi phép toán **khả vi** (differentiable), nên PyTorch có thể tự động tính đạo hàm của loss theo bất kỳ tham số nào bằng quy tắc chuỗi (chain rule):

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_x \frac{\partial \mathcal{L}}{\partial C(x)} \cdot \frac{\partial C(x)}{\partial \theta}, \quad \theta \in \{\mu, s, q, \alpha, k_{\ell m}\}$$

- Với mỗi pixel x mà một Gaussian có góp mặt, đạo hàm $\partial C(x)/\partial \theta$ lan ngược qua đúng chuỗi các bước đã tính thuận: alpha blending $\rightarrow \alpha_n(x) = \min(0.99, \alpha_n G_n(x)) \rightarrow G_n(x) \rightarrow \Sigma', \mu'$ (EWA) $\rightarrow \Sigma, \mu$ (không gian 3D) $\rightarrow s, q$ (tham số hoá hiệp phương sai).
- **Quan sát then chốt cho phần sau:** gradient của **vị trí chiếu** $\partial \mathcal{L}/\partial \mu'$ (trong không gian màn hình) có xu hướng **tập trung mạnh ở vùng biên** của Gaussian trong ảnh — nơi cường độ ảnh thay đổi nhanh nhất — chứ không rải đều khắp cả footprint. Điều này hợp lý về trực giác: ở vùng phẳng, sai một chút vị trí Gaussian không ảnh hưởng nhiều tới màu render; ở biên/cạnh sắc, một sai lệch nhỏ về vị trí gây thay đổi màu đáng kể ngay lập tức.
- Chính đại lượng $\partial \mathcal{L}/\partial \mu'$ này là **tín hiệu** mà Adaptive Density Control dùng để quyết định "Gaussian này có đang thiếu/thừa hình học không" (2 slide sau) — nhưng bản thân tín hiệu này cũng ẩn chứa một vấn đề tinh vi, sẽ nói ngay sau đây.



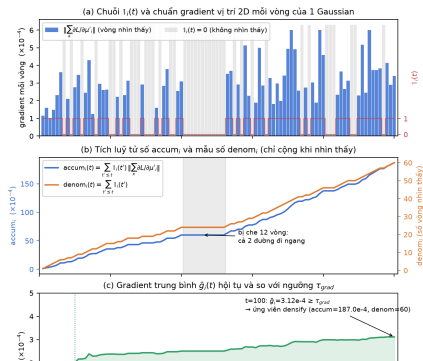
Tích lũy gradient qua nhiều view: lấy CHUẨN trước, cộng sau

Một Gaussian thường được camera train nhìn thấy từ **nhiều góc khác nhau** trong một epoch (một lượt qua hết mọi camera). Để quyết định densify, cần tổng hợp gradient thu được từ tất cả các lần render đó. Hàm `add_densification_stats` (`scene/gaussian_model.py:1054--1057`) làm điều này như sau:

$$\text{xyz_gradient_accum}_i += \|\nabla_{\mu'_i} \mathcal{L}\|_2, \quad \bar{g}_i = \frac{\text{xyz_gradient_accum}_i}{\text{denom}_i}$$

với denom_i là số lần Gaussian i được nhìn thấy (và cộng dồn) tính tới thời điểm đó.

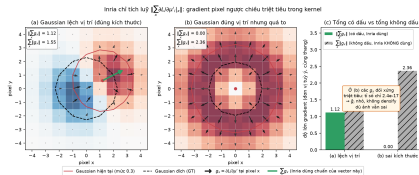
- **Điểm mấu chốt cần nhớ:** phép **lấy chuẩn** $\|\cdot\|$ diễn ra **ngay tại mỗi lần render** (mỗi view), **trước khi** cộng dồn qua denom — không phải cộng các *vector* gradient của nhiều view lại với nhau rồi mới lấy chuẩn một lần ở cuối.
- Hệ quả trực tiếp: vì mỗi số hạng được cộng vào `xyz_gradient_accum` luôn là một **số không âm** (chuẩn của một vector luôn ≥ 0), gradient từ các view **khác nhau tuyệt đối không thể triệt tiêu lẫn nhau** trong phép cộng này.
- Repo này còn tích lũy song song một kênh **trị tuyệt đối** riêng biệt, `xyz_gradient_accum_abs` — dùng làm tín hiệu cho ngưỡng `grad_abs_thresh` khi quyết định **split**, tách biệt hoàn toàn với ngưỡng `grad_thresh` dùng cho **clone**. Vì sao cần tới hai kênh gradient khác nhau — xem slide kế tiếp.



Vấn đề "triệt tiêu gradient": xảy ra GIỮA CÁC PIXEL, không phải giữa các view

Cách hiểu sai thường gặp: "gradient từ các camera khác nhau kéo ngược chiều nhau nên triệt tiêu lẫn nhau". Cách hiểu này **sai** với chính pipeline đã trình bày — vì chuẩn $\|\cdot\|$ đã được lấy **trước khi** cộng qua các view (slide trước), phép cộng *giữa các view* không thể nào cho ra một tổng nhỏ hơn từng số hạng.

- **Sự thật:** hiện tượng triệt tiêu thật sự xảy ra *bên trong một lần render duy nhất, giữa các pixel* thuộc cùng footprint của **một** Gaussian. Ví dụ: nếu ảnh thật có một cạnh chạy ngang qua giữa Gaussian, nửa pixel bên trái có thể "kéo" μ' sang trái để khớp tốt hơn, nửa bên phải lại "kéo" sang phải — khi **cộng vector gradient của từng pixel lại** (trước khi lấy chuẩn, ngay trong một lần render), hai lực kéo ngược chiều này có thể gần như **triệt tiêu nhau**, cho ra một vector tổng rất nhỏ dù bản thân mỗi pixel đều có gradient đáng kể.
- **Hệ quả nguy hiểm:** một Gaussian đang thực sự "dưới tái tạo" (thiếu chi tiết hình học ở đó) nhưng đối xứng theo một trục nào đó có thể bị đánh giá nhầm là "gradient thấp $\Rightarrow$ không cần densify" — một điểm mù của cơ chế chỉ dùng gradient có dấu.
- **Cách khắc phục (kỹ thuật AbsGS):** renderer CUDA tính thêm, ngay trong backward pass, một kênh gradient **trị tuyệt đối theo từng pixel trước khi** các dấu đối lập có cơ hội triệt tiêu nhau — chính là kênh `xyz_gradient_accum_abs` đã nhắc ở slide trước.



Adaptive Density Control (ADC): khung chung

Có gradient tích lũy $\bar{g}$ (và $\bar{g}_{abs}$) rồi, cứ mỗi `densification_interval` = 100 iteration (bắt đầu từ `densify_from_iter` = 500, tức lần đầu ở iteration 600, tới `densify_until_iter` = 15 000), 3DGS thực hiện 3 loại thao tác:

Thao tác	Điều kiện kích hoạt (ý tưởng)	Mục đích
Clone	gradient cao, Gaussian đang <i>nhỏ</i>	bù <i>under-reconstruction</i> (thiếu hình học)
Split	gradient cao, Gaussian đang <i>to</i>	bù <i>over-reconstruction</i> (Gaussian quá to, làm mờ chi tiết)
Prune	opacity quá thấp, hoặc quá to	dọn Gaussian không còn đóng góp

"To/nhỏ" được so với ngưỡng `percent_dense × extent` (scene extent đã định nghĩa ở phần đầu): Gaussian có trục dài nhất $\leq$ ngưỡng này thì thuộc diện **clone**, lớn hơn thì thuộc diện **split**.

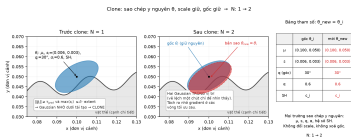
- Trực giác: một Gaussian **nhỏ** nhưng vẫn còn gradient cao có nghĩa là "còn thiếu chi tiết ở đây, cần thêm quân số" → nhân bản. Một Gaussian **to** mà vẫn còn gradient cao nghĩa là "đang cố khớp một vùng có chi tiết mà một Gaussian không đủ" → chia nhỏ ra.

Clone: sao chép y nguyên, KHÔNG lệch vị trí

Khi một Gaussian nhỏ có gradient vượt ngưỡng, thao tác `densify_and_clone` thực hiện điều tương chừng đơn giản đến bất ngờ: **sao chép y hệt** mọi tham số sang một Gaussian con mới — **cùng vị trí**, cùng scale, cùng rotation, cùng màu:

$$\theta_{\text{con}} = \theta_{\text{cha}} \quad (\text{không có bất kỳ } + \varepsilon \text{ hay lấy mẫu ngẫu nhiên nào})$$

- **Câu hỏi tự nhiên:** nếu hai bản sao giống hệt nhau, làm sao chúng tách ra để thực sự bổ sung chi tiết thay vì vẫn cứ trùng khít mãi? **Trả lời:** ngay sau khi sinh ra, hai Gaussian đi qua **optimizer độc lập** (mỗi Gaussian có trạng thái Adam — `exp_avg`, `exp_avg_sq` — riêng của nó). Ở những bước gradient descent tiếp theo, dù xuất phát từ cùng một điểm, hai Gaussian này chịu nhiều số học và các tương tác nhỏ khác nhau với các Gaussian lân cận $\Rightarrow$ dần dần bị “đẩy” ra hai hướng khác nhau một cách tự nhiên, giúp phủ tốt hơn vùng đang thiếu chi tiết.
- Các bộ đếm thống kê (`xyz_gradient_accum`, `denom`, v.v.) của bản sao mới được **reset về 0** ngay sau khi sinh — lịch sử theo dõi của nó hoàn toàn tách biệt với Gaussian cha kể từ thời điểm này.



Split nguyên bản: lấy mẫu ngẫu nhiên quanh Gaussian cha

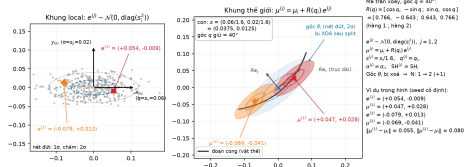
Khi một Gaussian **to** (trục dài nhất > ngưỡng `percent_dense × extent`) vẫn còn gradient trị tuyệt đối cao, cơ chế **split nguyên bản** (`densify_and_split`, `gaussian_model.py:866--889`) sinh ra $N = 2$ Gaussian con để "chia việc" khớp chi tiết:

$$x_{\text{con}} = \mu_{\text{cha}} + R(q) \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \text{diag}(s_{\text{cha}})^2), \quad s_{\text{con}} = \frac{s_{\text{cha}}}{0.8 N} = \frac{s_{\text{cha}}}{1.6}$$

- Vị trí mỗi con được **lấy mẫu ngẫu nhiên** từ một phân phối Gaussian có độ lệch chuẩn chính bằng scale của cha, sau đó xoay theo hướng $R(q)$ của cha — tức là các con rải rác quanh cha, theo đúng hình dạng ellipsoid của cha (không rải đều hình cầu).
- Cả $N = 2$ con đều bị chia scale cho **cùng một hệ số 1.6 trên cả 3 trục**, bất kể trục nào mới thực sự là trục "quá to" gây ra vấn đề — đây là điểm mà cơ chế split dị hướng của SADGS (phần sau bài giảng) cải tiến.
- Gaussian cha bị xóa ngay sau khi sinh 2 con — tổng số Gaussian tăng thêm đúng 1 đơn vị mỗi lần một điểm bị split.

Split: Gaussian TO ($\max(s_i) > \delta \cdot \text{extent}$) gán một đoạn con $\Rightarrow 2$ con nhỏ hơn 1.6x

Công thức split (2D)



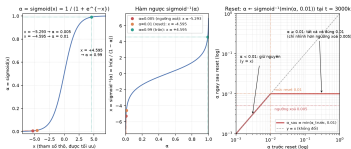
Vì sao lưu opacity dưới dạng logit? Và cơ chế reset định kỳ

Opacity $\alpha \in (0, 1)$ được lưu nội bộ dưới dạng **logit** $o = \sigma^{-1}(\alpha) = \ln \frac{\alpha}{1-\alpha}$ (giá trị thực bất kỳ), rồi đi qua sigmoid $\alpha = \sigma(o)$ khi cần dùng — giống hệt lý do dùng exp cho scale: để mọi số thực đều tương ứng với một giá trị α **hợp lệ**, không cần ràng buộc clamp thủ công trong quá trình tối ưu bằng gradient descent.

- **Vì sao cần reset opacity định kỳ?** Nếu để tự nhiên, nhiều Gaussian "chết dần" (opacity giảm về rất thấp) vẫn tồn tại lâu, chiếm bộ nhớ mà không đóng góp gì đáng kể. Mỗi `opacity_reset_interval = 3000` iteration (với điều kiện `iteration < densify_until_iter = 15 000`, tức **chỉ đúng 4 lần**: tại 3k, 6k, 9k, 12k), toàn bộ opacity bị **giảm mạnh**:

$$\alpha \leftarrow \alpha_{\text{cũ}} \times \text{opacity_reset_decay} = \alpha_{\text{cũ}} \times 0.1$$

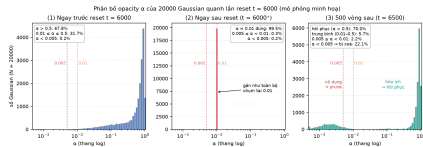
- **Cơ chế "sàng lọc"**: Gaussian nào thực sự hữu ích (đang đóng góp giảm loss) sẽ nhanh chóng **hồi phục** opacity qua vài trăm bước gradient tiếp theo; Gaussian nào không còn đóng góp thì opacity vẫn ở mức thấp và sẽ bị prune ở đợt dọn dẹp kế tiếp (slide sau).
- Vì reset **dừng hẳn** sau iteration 15 000, nửa sau của quá trình train không còn cơ chế chủ động nào "sàng lọc lại" opacity định kỳ nữa.



Nhìn trên toàn bộ quần thể: phân bố opacity trước và sau reset

Thay vì theo dõi một Gaussian, hãy nhìn **toàn bộ phân bố** opacity của mọi Gaussian ngay trước và ngay sau một lần reset. Ngay sau reset, toàn bộ phân bố bị "nén" mạnh về phía các giá trị thấp (do nhân với 0.1) — gần như mọi Gaussian đều tạm thời "mờ đi" đồng loạt.

- Điều thú vị xảy ra ở các iteration **sau** reset: phân bố dần **tách thành hai nhóm** — một nhóm hồi phục nhanh trở lại vùng opacity cao (Gaussian hữu ích), một nhóm tiếp tục nằm lại vùng thấp (ứng viên prune).
- Đây chính là hiệu ứng "sàng lọc" đã mô tả ở hai slide trước, nhưng nhìn dưới góc độ **thống kê quần thể** thay vì một cá thể — một cách trực quan để kiểm chứng cơ chế reset có thực sự đang "phân loại" Gaussian hữu ích/dư thừa hay không trong một lần chạy huấn luyện cụ thể.



Tổng kết: pipeline 3DGS gốc, đầu-cuối

Gộp lại toàn bộ phần vừa trình bày thành một vòng lặp duy nhất:

$$\text{SfM/COLMAP} \rightarrow \{\mu, \Sigma, \alpha, c\}_{i=1}^N \xrightarrow{\text{EWA}} \text{ellipse 2D} \xrightarrow{\text{compact box}} \text{tile} \xrightarrow{\text{blend}} C(x) \xrightarrow{\mathcal{L}} \nabla \theta \xrightarrow{\text{Adam}} \theta_{\text{mới}}$$

lặp lại hàng chục nghìn lần, với riêng mỗi 100 iteration lại chen thêm bước **ADC** (clone/split/prune) dựa trên gradient tích lũy, và mỗi 3000 iteration (tới 15 000) lại reset opacity một lần.

- **Điểm mạnh của thiết kế này:** hoàn toàn tường minh và khả vi từ đầu đến cuối — không hộp đen, mọi bước đều có thể kiểm tra, gỡ lỗi; render thời gian thực ngay sau khi train xong.
- **Điểm còn hạn chế, là động lực cho phần tiếp theo của bài giảng:** tín hiệu densify **chỉ dựa vào gradient** — một đại lượng phản ánh *trạng thái hội tụ hiện tại* của loss, chứ không trực tiếp đo “ảnh này còn thiếu bao nhiêu % chi tiết tần số cao chưa được biểu diễn, theo tỉ lệ nào”. Hai Gaussian có cùng độ lớn gradient có thể đang thiếu hụt hình học ở hai *quy mô* rất khác nhau (một cái thiếu chi tiết rất mịn, một cái thiếu cấu trúc thô hơn nhiều) — gradient một mình không phân biệt được điều này.



Hình: Ảnh render minh họa (định tính, không kèm số đo benchmark).

SADGS: bổ sung tín hiệu tần số cho Adaptive Density Control

Nhắc lại hạn chế đã nêu ở cuối phần trước: gradient một mình không đo được "Gaussian này đang to/nhỏ so với chi tiết ảnh cần biểu diễn theo tỉ lệ nào".
Ý tưởng cốt lõi của SADGS (Structure-Aware Densification): đo trực tiếp một tỉ số hình học giữa **kích thước Gaussian sau khi chiếu lên màn hình** và **kích thước chi tiết ảnh cục bộ** (một dạng "bước sóng" texture) tại đúng vị trí Gaussian đó — gọi tỉ số này là η .

- Nếu Gaussian **to hơn** chi tiết cần biểu diễn ($\eta > 1$): nó đang "nuốt chửng", làm mờ mất chi tiết đó — cần **split** thành các mảnh nhỏ hơn, và quan trọng hơn: chia **đúng theo trục** bị vi phạm, không phải chia đều mọi trục như cơ chế gốc.
- Nếu Gaussian **nhỏ hơn nhiều** so với chi tiết cục bộ ($\eta \leq 0.1$): nó đang "phục vụ thừa" một vùng gần như không đổi — một Gaussian lớn hơn cũng đủ sức biểu diễn vùng đó — là ứng viên **prune**.
- **Điểm quan trọng cần nhấn mạnh**: SADGS **không thay thế** khung ADC gốc vừa học — nó **kế thừa toàn bộ** khung clone/split/prune, chỉ **bổ sung thêm** một điều kiện kích hoạt dựa trên η , kết hợp bằng phép OR với điều kiện gradient cũ (chi tiết đầy đủ ở phần sau).
- Bốn slide tiếp theo xây dựng η từ con số 0: ảnh gốc $\rightarrow$ kim tự tháp đa tỉ lệ $\rightarrow$ structure tensor $\rightarrow$ trị riêng $\rightarrow$ bước sóng cục bộ $\rightarrow$ chiếu trục Gaussian lên màn hình $\rightarrow \eta$.

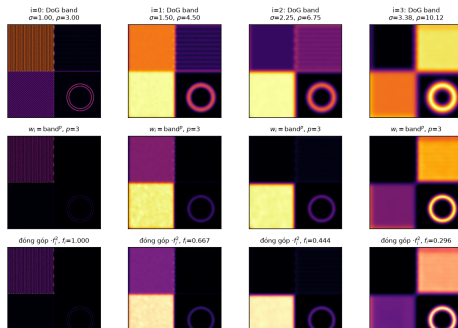
Bước 1: kim tự tháp đa tỉ lệ (multiscale) trên ảnh GT

Trước khi có thể so sánh "kích thước Gaussian" với "kích thước chi tiết ảnh", cần một cách đo **chi tiết ảnh có kích thước bao nhiêu** tại mỗi vị trí. Hàm `get_multiscale_structure_tensor_v1` (utils/loss_utils.py:232--) làm mờ dần ảnh gốc qua nhiều tầng $i = 0, \dots, \text{levels} - 1$ (mặc định 3 tầng), tầng sau mờ hơn tầng trước theo một tỉ lệ cố định:

$$\sigma_i = \sigma_0 \cdot (\text{octave_step})^i, \quad \text{octave_step} = 1.5, \quad b_i = \|I_{\text{mờ}}^{(i-1)} - I_{\text{mờ}}^{(i)}\|_2 \quad (\text{DoG, xấp xỉ Laplacian})$$

- b_i (*Difference-of-Gaussians*, kỹ thuật kinh điển trong thị giác máy tính, ví dụ dùng trong SIFT): là **bản đồ năng lượng cạnh** tại tầng i — vùng phẳng cho $b_i \approx 0$, vùng có cạnh/chi tiết rõ cho b_i lớn.
- Tần số ứng với mỗi tầng: $f_i = 1/(\text{octave_step})^i$ — tầng i càng lớn (mờ càng nhiều) thì f_i càng nhỏ, ứng với chi tiết **thô** hơn.
- **Vì sao cần nhiều tỉ lệ chứ không chỉ một?** Một hoạ tiết lặp (ví dụ vân gỗ, gạch lát) có thể "rõ" khi nhìn ở độ phân giải mịn nhưng gần như "biến mất" khi nhìn ở độ phân giải thô, hoặc ngược lại với cấu trúc lớn (viên một toà nhà). Gộp nhiều tầng tránh bỏ sót chi tiết chỉ tồn tại rõ ràng ở một quy mô riêng.

Kim tự tháp v2: levels=4, base_sigma=1, octave_step=1.5, power_factor=3, smoothing_factor=1
 $\sigma_i = \sigma_0 s^i, \quad f_i = s^{-i}, \quad \rho_i = 3\sigma_i$



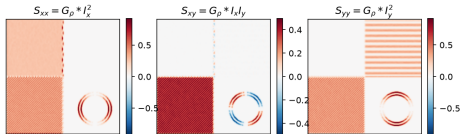
Bước 2: gộp thành một structure tensor duy nhất $\hat{J}$

Tại mỗi tầng i , ta có thể tính một **structure tensor** $J_i = \begin{pmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{pmatrix}_i$ — một ma trận 2×2 cổ điển trong xử lý ảnh, đo **hướng** và **cường độ** biến thiên cục bộ (dựng từ tích ngoài của vector gradient ảnh, làm mờ nhẹ để ổn định). Structure tensor gộp toàn bộ các tầng theo trọng số (`loss_utils.py:283--300`):

$$\hat{J} = \frac{\sum_i (J_i / \text{tr} J_i) w_i f_i^2}{\sum_i w_i}, \quad w_i = b_i^p \quad (p = 3, \text{ mặc định})$$

- $J_i / \text{tr} J_i$: **chuẩn hoá theo vết** — mỗi tầng chỉ còn đóng góp thông tin *hướng/hình dạng* (dị hướng của biến thiên), loại bỏ ảnh hưởng của việc tầng nào đó có năng lượng tuyệt đối lớn hơn tầng khác một cách tình cờ.
- $w_i = b_i^3$: tầng nào có cạnh/chi tiết rõ ràng hơn (b_i lớn hơn) được **"tin cậy" nhiều hơn** khi gộp; luỹ thừa 3 khuếch đại sự chênh lệch này khá mạnh, để tầng "nhiều" (ít cấu trúc) không làm loãng thông tin từ tầng có cấu trúc rõ.
- f_i^2 : hệ số quy đổi để các tầng — vốn bị làm mờ ở các σ_i khác nhau — được đưa về **cùng một thang tần số vật lý** trước khi cộng gộp.
- Kết quả $\hat{J} = (S_{xx}, S_{xy}, S_{yy})$ chính là "nguyên liệu" dùng lại xuyên suốt các bước tính η tiếp theo.

$J_p = G_p * \nabla I / |\nabla I|^T$ — chuẩn hoá bởi $\max(S_{xx} + S_{yy})$ (`loss_utils.py:222-227`)

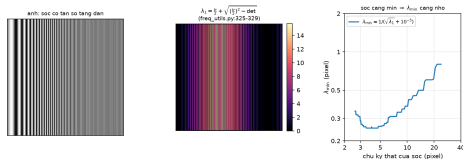


Bước 3: trị riêng của $\hat{J} \rightarrow$ bước sóng cục bộ $w_{\min}$

Từ ma trận $\hat{J}$, trích ra **trị riêng lớn nhất** λ_1 (utils/freq_utils.py:313--334) — đại lượng vô hướng duy nhất cần cho bước tiếp theo:

$$\lambda_1 = \frac{\text{tr}\hat{J}}{2} + \sqrt{\left(\frac{\text{tr}\hat{J}}{2}\right)^2 - \det\hat{J}}, \quad w_{\min} = \frac{1}{\sqrt{\lambda_1} + 10^{-5}}$$

- λ_1 tỉ lệ với **năng lượng gradient bình phương** theo hướng biến thiên mạnh nhất tại điểm đó — vùng có cạnh sắc, chi tiết dày đặc sẽ cho λ_1 lớn; vùng phẳng cho $\lambda_1 \approx 0$.
- Phép biến đổi "năng lượng $\rightarrow$ bước sóng" dùng nghịch đảo căn bậc hai: trực giác là năng lượng gradient cao tương ứng với **tần số không gian cao** (chi tiết mịn, đổi nhanh theo pixel), mà tần số cao nghĩa là **bước sóng ngắn** — do đó λ_1 lớn $\Rightarrow w_{\min}$ **nhỏ**. Ngược lại, vùng phẳng $\lambda_1 \rightarrow 0$ cho $w_{\min} \rightarrow 10^5$ (rất lớn — "không có chi tiết nào cần bám theo ở đây cả").
- $w_{\min}$ sẽ đóng vai trò **mẫu số**, là "cây thước" để so sánh với kích thước Gaussian ở bước tính η ngay sau đây.

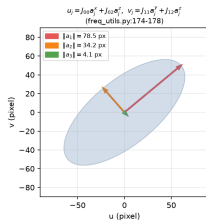
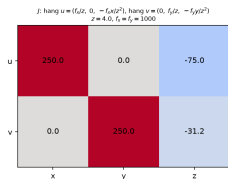


Bước 4: chiếu 3 trục chính của Gaussian lên màn hình

Có "cây thước" $w_{\min}$ rồi, giờ cần đo "kích thước Gaussian" theo đúng đơn vị tương thích (pixel trên màn hình). Hàm `compute_projected_axes_subset` (`utils/freq_utils.py:116--178`) dùng lại đúng Jacobian J của phép chiếu pinhole đã học ở phần đầu bài giảng:

$$a_j = J(R_{\text{view}} R_i) \text{diag}(s) e_j, \quad \ell_j = \|a_j\| = \sqrt{u_j^2 + v_j^2 + 10^{-8}}$$

- $R_i \text{diag}(s) e_j$: véc-tơ trục chính thứ j của ellipsoid ($j = 1, 2, 3$, ba trục vuông góc của ellipsoid trong không gian world), có độ dài = s_j .
- R_{view} xoay véc-tơ đó sang hệ toạ độ camera; J tuyến tính hoá phép chiếu phối cảnh, đánh giá tại đúng vị trí μ của Gaussian — giống hệt cách dùng J trong EWA splatting ở phần render.
- ℓ_j là **độ dài tính bằng pixel**, sau khi chiếu, của trục chính thứ j — đây chính là tử số của η .
- Mỗi Gaussian có **3 giá trị** ℓ_1, ℓ_2, ℓ_3 (ứng với 3 trục chính hình học, **không phải** 3 kênh màu R/G/B) — nhờ đó SADGS biết được Gaussian đang vi phạm *theo hướng cụ thể nào*, để sau này split *đúng trục* đó thay vì chia đều mọi hướng.



Bước 5: $\eta = \ell/w_{\min}$ — trực giác Nyquist

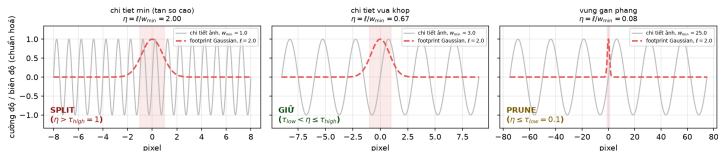
Giờ đã có cả tử số (ℓ , kích thước Gaussian đã chiếu) lẫn mẫu số ($w_{\min}$, bước sóng chi tiết cục bộ), định nghĩa tỉ số cấu trúc:

$$\eta_j = \frac{\ell_j}{w_{\min}} \quad (\text{eta_compute_mode}=\text{"wavelength"}, \text{mặc định, freq_utils.py:348})$$

Đây là một ngưỡng **hình học thuần túy** (so sánh hai độ dài), **không phải** một mô hình vật lý về suy hao biên độ tín hiệu — tên gọi “trực giác Nyquist” chỉ mượn ý tưởng chung của lý thuyết lấy mẫu: muốn biểu diễn đúng một dao động, “đơn vị lấy mẫu” (ở đây là Gaussian) phải đủ nhỏ so với chu kỳ của dao động đó.

- $\eta > 1$: ℓ **rộng hơn cả một chu kỳ** chi tiết $\Rightarrow$ một Gaussian không đủ khả năng bám theo dao động đó $\Rightarrow$ ứng viên **SPLIT**.
- $0.1 < \eta \leq 1$: kích thước tương xứng với chi tiết cần biểu diễn $\Rightarrow$ **GIỮ NGUYÊN**.
- $\eta \leq 0.1$: ℓ nhỏ hơn $w_{\min}$ tới hơn 10 lần — Gaussian đang “phủ thừa” một vùng gần như không đổi $\Rightarrow$ ứng viên **PRUNE**.
- **Lưu ý về tài liệu nội bộ code**: một số dòng docstring (gaussian_model.py:837--840) mô tả $\eta \sim (\sigma\omega)^2$ (quan hệ *bậc hai*) — nhưng công thức **thực sự được tính và dùng** ở trên là quan hệ **tuyến tính** theo kích thước. Toàn bộ bài giảng này bám theo công thức *thực chạy*, không theo docstring (vốn có thể là tài liệu chưa cập nhật theo code).

$\eta = \ell/w_{\min}$: so sánh KÍCH THƯỚC Gaussian với bước sóng cục bộ (không phải bộ lọc suy hao)



Bước 6: ngưỡng phân loại và tích lũy NHẤT QUÁN ĐA VIEW

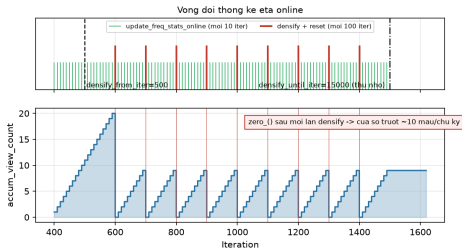
Ngưỡng cố định trong code: `TAU_HIGH=1.0`, `TAU_LOW=0.1` (`freq_utils.py:395--396`). Mỗi lần render một view, lấy $\eta_{\max} = \max_j \eta_j$ (giá trị lớn nhất trong 3 trục) và cộng 1 vào đúng một trong ba bộ đếm:

$$\text{eta_high_count} \ (\eta_{\max} > 1) \quad \text{eta_mid_count} \quad \text{eta_low_count} \ (\eta_{\max} \leq 0.1)$$

Tới kỳ densify (mỗi 100 iteration), các bộ đếm này được quy ra **tỉ lệ nhất quán qua nhiều view** (`train.py:341--353`) — đây chính là điểm khác biệt lớn nhất về *triết lý* so với 3DGS gốc, vốn chỉ nhìn vào gradient tích lũy tại **một** thời điểm cuối cùng để quyết định, không phân biệt rõ ràng “có bao nhiêu góc nhìn đồng ý với nhau”:

$$\text{high_ratio} = \frac{\text{eta_high_count}}{\text{accum_view_count}} > 0.8 \quad \text{low_ratio} = \frac{\text{eta_low_count}}{\text{accum_view_count}} > 0.8$$

- **Trực giác:** chỉ khi **đa số áp đảo** ($> 80\%$) các góc nhìn quan sát được đều đồng thuận rằng Gaussian “quá to” (hoặc “quá nhỏ”), quyết định split/prune mới được kích hoạt — giúp tránh phản ứng thái quá theo nhiễu của một góc chụp bất thường (ví dụ do occlusion, ánh sáng loá, hay góc nhìn gần như cạnh của ellipsoid).

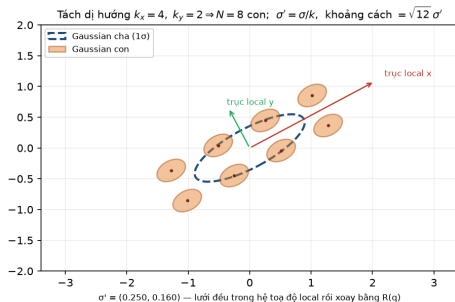


Split dị hướng (1/2): chia đúng trục, đặt con trên lưới tất định

densify_and_split_structgs (gaussian_model.py:638--) — phiên bản split **mới** của SADGS, khi có sẵn max_eta_3ch, tính riêng cho từng trục:

$$k_{\text{axis}} = \lceil \sqrt{\max(\eta_{\text{axis}}, 1)} \rceil, \quad N_{\text{con}} = k_x k_y k_z, \quad s_{\text{con}} = \frac{s_{\text{cha}}}{k_{\text{axis}}^p} \quad (p = \text{ks_scale_power} = 1.0)$$

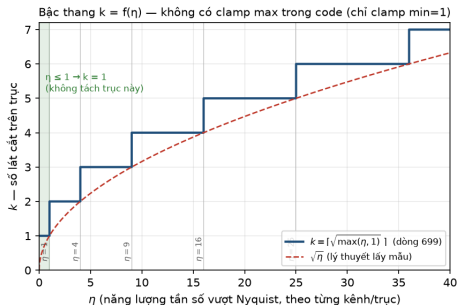
- Điểm khác biệt lớn với split nguyên bản (slide trước, phần 1): k được tính **riêng cho từng trục** — trục nào thực sự vi phạm ($\eta_{\text{axis}} > 1$) mới bị chia; trục còn "khoẻ mạnh" giữ nguyên $k = 1$.
- Vị trí các con được đặt trên một **lưới tất định** (deterministic grid) — **không** lấy mẫu ngẫu nhiên như cơ chế gốc: khoảng cách tâm-tâm giữa các con liền kề $= \sigma_{\text{con}} \sqrt{12}$, toàn bộ lưới được xoay theo đúng hướng R của Gaussian cha (gaussian_model.py:766--780).
- Có thể kiểm chứng lưới này **bảo toàn phương sai**: tổng phương sai của các con đặt cách đều theo công thức trên đúng bằng phương sai của cha ban đầu — tức đây không phải một lựa chọn tùy ý mà có cơ sở thống kê.



Split dị hướng (2/2): k tăng theo bậc thang khi η tăng

Vì $k_{\text{axis}} = \lceil \sqrt{\max(\eta_{\text{axis}}, 1)} \rceil$ là một hàm **làm tròn lên**, quan hệ giữa k và η không mượt mà là một **dạng bậc thang**: k chỉ nhảy lên một mức mới khi η vượt qua các ngưỡng 1, 4, 9, 16, ... (k^2). Điều này có nghĩa:

- Với η vừa mới vượt 1 (ví dụ $\eta = 1.2$): $k = 2$, sinh $N_{\text{con}} = 2$ trên trục đó — một mức tăng “thận trọng”, tương xứng với mức độ vi phạm còn nhẹ.
- Với η lớn hơn nhiều (ví dụ $\eta = 20$): $k = \lceil \sqrt{20} \rceil = 5$, sinh hẳn 5 con trên trục đó — phản ứng **mạnh tay hơn** tương xứng với mức độ vi phạm nghiêm trọng hơn.
- So với split nguyên bản (luôn cố định $N = 2$ bất kể mức độ “quá to” ra sao), cách làm bậc thang này cho phép hệ thống **tự điều chỉnh cường độ phản ứng** theo đúng quy mô của vấn đề — tránh vừa chia không đủ (còn dư $\eta > 1$ sau một lần split) vừa tránh chia quá tay gây bùng nổ số lượng Gaussian không cần thiết.

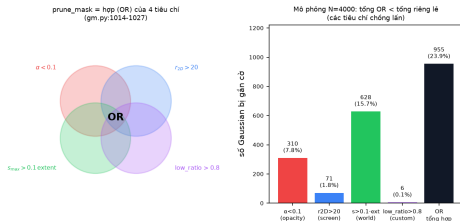


Prune theo cấu trúc: OR với tiêu chí gốc, không thay thế

Trong `densify_and_prune_structgs` (`gaussian_model.py:1009--1019`), mặt nạ prune **cấu trúc** (`low_ratio > 0.8`, slide trước) được kết hợp bằng phép **OR** với các tiêu chí nguyên bản đã học ở phần ADC gốc — một Gaussian bị xóa nếu **bất kỳ** điều kiện nào sau đúng:

$$\text{prune_mask} = \underbrace{(\alpha < 0.1)}_{\text{opacity thấp}} \vee \underbrace{(r_{2D} > \text{max_screen_size})}_{\text{quá to trên màn hình}} \vee \underbrace{(\max(s) > 0.1 \cdot \text{extent})}_{\text{quá to trong không gian}} \vee \underbrace{(\text{low_ratio} > 0.8)}_{\text{cấu trúc, đa view}}$$

- `min_opacity= 0.1` trong nhánh này — **cao hơn nhiều** so với 0.005 ở nhánh 3DGS gốc, tức prune mạnh tay hơn theo tiêu chí opacity đơn thuần, không chỉ nhờ thêm điều kiện cấu trúc.
- Cả clone lẫn split (2 slide trước) còn bị chặn thêm bởi `metric_mask= (accum_view_count > 0.5)` — một Gaussian vừa mới sinh, chưa từng được quan sát trong đợt tích lũy hiện tại, sẽ **không** bị động tới ở đợt densify đó, tránh phản ứng vội vàng dựa trên dữ liệu quá ít ỏi.



Luồng quyết định đầy đủ: densify_and_prune_structgs

Đây là hàm **thật sự đang chạy mặc định** mỗi 100 iteration (`gaussian_model.py:957--1050`), ghép tín hiệu η (5 slide trước) **OR** với tín hiệu gradient (đã học ở phần ADC gốc) thành một luồng quyết định duy nhất:

$$\text{split_qual} = \max(s) > \text{dense} \cdot \text{extent}, \quad \text{clone_qual} = \max(s) \leq \text{dense} \cdot \text{extent} \quad (\text{dense} = 0.001)$$

$$\text{final_split} = (\text{split_mask}_\eta \vee \|\bar{g}_{\text{abs}}\| \geq \text{grad_abs_thresh}) \wedge \text{split_qual}$$

$$\text{final_clone} = (\text{split_mask}_\eta \vee \|\bar{g}\| \geq \text{grad_thresh}) \wedge \text{clone_qual}$$

với `grad_thresh=grad_abs_thresh= 0.0002` trong nhánh này (khác với cặp giá trị 0.0002/0.0004 chỉ dùng ở nhánh 3DGS gốc). Cả hai luồng còn bị AND thêm với `metric_mask` (đã học ở slide trước) rồi mới thực thi. Sau clone/split: chạy prune (đã học), rồi áp **trần opacity** $\alpha \leftarrow \min(\alpha, 0.8)$ lên **toàn bộ** Gaussian còn lại — giới hạn opacity tối đa để tránh một vài Gaussian trở nên quá "đặc", che khuất hoàn toàn các Gaussian phía sau và cản trở gradient của chúng.

Đâu là điểm MỚI của SADGS, đâu là KẾ THỪA — trong số những gì ĐANG CHẠY

Cơ chế đang chạy mặc định	Nguồn gốc
Compact box (SnugBox, $\text{mult}=0.7$)	kế thừa (Taming/Speedy-Splat)
Bộ lọc low-pass 2D ($\kappa I, \kappa = 0.1$)	kế thừa (Zwicker et al., EWA splatting)
Optimizer hybrid (Adam thường + Sparse/Fused Adam)	kế thừa (Taming), mặc định , không tắt
Structure tensor đa tỉ lệ $\rightarrow$ bước sóng $w_{\min} \rightarrow \eta$	mới (SADGS)
Tích lũy nhất quán đa view ($\text{high_ratio/low_ratio}$)	mới (SADGS)
Split dị hướng theo từng trục (k_x, k_y, k_z riêng biệt)	mới (SADGS)
Prune theo cấu trúc (kết hợp OR với tiêu chí gốc)	mới (SADGS)

- **Thông điệp chính:** phần "mới" thực sự của SADGS nằm ở **toàn bộ chuỗi xây dựng và sử dụng η** — không phải ở compact box hay optimizer, vốn là các kỹ thuật tăng tốc **kế thừa** từ các công trình khác và đã có sẵn trước khi SADGS được thêm vào.
- Ba slide tiếp theo đi sâu vào phần **tối ưu hoá** (optimizer, learning rate, lịch reset) — các cơ chế này áp dụng như nhau cho cả nhánh 3DGS gốc lẫn nhánh SADGS, không phải điểm phân biệt giữa hai nhánh.

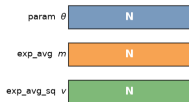
Optimizer: hybrid là MẶC ĐỊNH — chạy song song hai bộ tối ưu

arguments/_init_.py:128: self.optimizer_type = "hybrid". Mỗi iteration (train.py:429--434), **cả hai** dòng sau đều chạy:

- 1 gaussians.optimizer.step() — Adam **thường** (với amsgrad=True) cập nhật các nhóm tham số chính: xyz, opacity, scaling, rotation, f_dc.
 - 2 gaussians.shoptimizer.step(visible, radii.shape[0]) — một **Sparse/Fused Gaussian Adam** riêng, chỉ cập nhật tham số của các Gaussian **đang nhìn thấy được** trong view hiện tại (visible = radii>0) — tiết kiệm phép tính vì phần lớn Gaussian trong một cảnh lớn thường nằm **ngoài khung hình** của bất kỳ camera đơn lẻ nào tại một thời điểm.
- Hai chế độ khác cũng tồn tại trong code nhưng **không phải mặc định**: "default" (chỉ chạy optimizer_step với lịch cập nhật thưa dần $1 \rightarrow 32 \rightarrow 64$ iteration/lần) và "sparse_adam" (chỉ chạy phần sparse, bỏ hẳn Adam thường).
- **Kết luận thực dụng**: optimizer không phải là chỗ để so sánh 3DGS gốc với SADGS — cả hai nhánh dùng chung một cấu hình optimizer hybrid này.

cat_tensors_to_optimizer: ghép M Gaussian mới vào Adam (566-594)

TRƯỚC (N Gaussian)



SAU (N+M Gaussian)



$$\theta' = \text{cat}(\theta, \theta[M]) \quad m' = \text{cat}(m, \mathbf{0}_M) \quad v' = \text{cat}(v, \mathbf{0}_M)$$

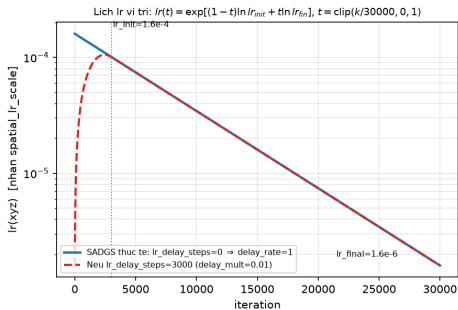
del opt.state[p] -> group["params"][0] = nn.Parameter(cat) -> opt.state[p_new] = state
(gaussian_model.py:578-586 — phải dùng lại param group vì id(Parameter) là khóa của opt.state)

Lịch learning rate: mỗi nhóm tham số một tốc độ riêng

Vị trí μ dùng lịch **suy giảm mũ có trễ khởi động**, từ $\text{position_lr_init} = 1.6 \times 10^{-4}$ giảm dần về $\text{position_lr_final} = 1.6 \times 10^{-6}$ qua 30 000 bước; các nhóm tham số còn lại dùng learning rate **cố định** suốt quá trình train:

Nhóm	lr	Nhóm	lr
opacity	0.05	rotation	0.002
scaling	0.01	f_dc (màu trung bình)	0.0025
f_rest (SH bậc cao)	0.00025		

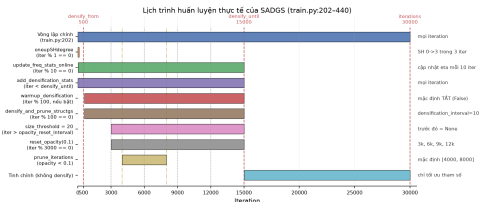
- **f_rest** (chi tiết màu phụ thuộc góc nhìn, SH bậc cao) học **chậm hơn 10 lần** f_dc (màu trung bình) — nguyên tắc chung: học các tham số "thô, quan trọng nền tảng" nhanh và ổn định trước, các tham số "tinh chỉnh chi tiết" học chậm và thận trọng hơn để tránh nhiễu loạn sớm.
- Vị trí μ được ưu tiên lr **lớn nhất lúc đầu** rồi giảm dần rất mạnh (giảm 100 lần qua 30 000 bước) — hình học cần thay đổi mạnh ở giai đoạn đầu để "định hình" cảnh, rồi ổn định dần khi đã gần hội tụ.



Xác nhận lại: lịch reset opacity đúng 4 lần, không hơn

Ghép lại tất cả các mốc thời gian liên quan tới reset opacity đã học (phần ADC gốc) thành một dòng thời gian trực quan: reset chỉ xảy ra khi `iteration < densify_until_iter = 15 000`, tại đúng bốn mốc 3000, 6000, 9000, 12000 — **không phải** một con số khác thường thấy trong một vài tài liệu tổng hợp trước đây (ví dụ "10 chu kỳ trải đều tới 30 000" — con số này **không khớp** với điều kiện `iteration < 15 000` nằm ngay trong đoạn code gọi `reset_opacity`).

- Việc xác nhận trực tiếp trên dòng thời gian thực tế (thay vì suy luận hay chép lại từ tài liệu khác) là một ví dụ cụ thể cho nguyên tắc xuyên suốt bài giảng này: **luôn đối chiếu với code thật** trước khi phát biểu một con số cụ thể nào đó.
- Cả nhánh 3DGS gốc lẫn nhánh SADGS đều dùng **chung** lịch reset này — đây không phải là điểm khác biệt giữa hai cơ chế densify.



Toàn cảnh lịch trình huấn luyện (30 000 iteration)

Mốc	Sự kiện	Ghi chú
iter 1 $\rightarrow$ 3	tăng bậc SH tới $D_{\max} = 3$	mỗi iteration
iter 500	bắt đầu densify	<code>densify_from_iter</code>
iter 600, 700, . . .	clone/split/prune (η OR gradient)	mỗi 100, tới iter 15 000
iter 3k, 6k, 9k, 12k	reset opacity ($\times 0.1$)	đúng 4 lần, chỉ khi $< 15\,000$
iter 4 000, 8 000	prune cứng thêm ($\alpha < 0.1$)	<code>prune_iterations</code>
iter 15 000	dừng hẳn densify/prune/reset	<code>densify_until_iter</code>
iter 15 000 $\rightarrow$ 30 000	chỉ còn tối ưu Adam, N không đổi	—

- **Không có** cơ chế nào chủ động thay đổi N sau iteration 15 000 — toàn bộ số lượng Gaussian đạt được tại mốc đó được giữ nguyên cho tới hết quá trình train, chỉ còn tối ưu giá trị tham số.
- Đây là điểm quan trọng khi so sánh 3DGS gốc với SADGS: khác biệt về **chất lượng/tốc độ hội tụ** chủ yếu xảy ra trong 15 000 iteration đầu, không phải "số Gaussian cuối cùng khác nhau nhờ tiếp tục tỉa về sau".

Chi phí tính toán: mô hình định tính, KHÔNG có số đo thật

Repo này **không chứa** script benchmark thời gian thực (wall-clock) nào đo trực tiếp và so sánh SADGS với 3DGS gốc trên cùng phần cứng, cùng scene, cùng số lần lặp lại đo. Một mô hình **định tính** hợp lý (không phải số đo thực nghiệm) cho chi phí mỗi iteration:

$$T_{\text{iter}} \approx aN + bNK + cN \cdot \mathbb{K}[\text{Adam step}] + F$$

với N số Gaussian, K số tile trung bình/Gaussian (bị chi phối bởi `mult` của compact box), F chi phí cố định mỗi khung hình (forward pass loss, sắp xếp, v.v.).

- Thành phần **có thể làm tăng** chi phí/iteration so với đường cơ sở: cập nhật η mỗi 10 iteration (thêm một lượt tính structure tensor và chiếu trục), hai optimizer chạy song song trong chế độ `hybrid`.
- Thành phần **có thể làm giảm** chi phí: `mult` = 0.7 < 1 giảm K so với SnugBox nguyên bản (`mult` = 1) — ít tile hơn cho mỗi Gaussian.
- **Nguyên tắc trình bày của bài giảng này:** bất kỳ con số "nhanh hơn X lần", "giây/iteration" nào không đi kèm phương pháp đo rõ ràng (phần cứng cụ thể, scene cụ thể, số lần lặp lại đo) sẽ **không được trích dẫn** như một kết quả benchmark đã kiểm chứng.

Tóm tắt: những gì ĐANG CHẠY mặc định trong repo này

Cơ chế	Vị trí trong code
Structure tensor đa tỉ lệ $\rightarrow \eta \rightarrow$ split/clone/prune	utils/freq_utils.py, train.py:276,341--372
Split dị hướng $k_x k_y k_z$ theo từng trục	scene/gaussian_model.py:638
Tích lũy nhất quán đa view (high/low ratio)	train.py:341--353
Reset opacity $\times 0.1$, đúng 4 lần	scene/gaussian_model.py:415
Trần opacity min(α , 0.8)	scene/gaussian_model.py:1043
Optimizer hybrid (Adam thường + Sparse/Fused Adam)	train.py:429--434
Compact box SnugBox, <code>mult= 0.7</code>	cuda_rasterizer/auxiliary.h:313
Bộ lọc low-pass 2D, $\kappa = 0.1$	cuda_rasterizer/forward.cu:113

Đây là danh sách đầy đủ những gì bài giảng này đã trình bày chi tiết — các nhánh tùy chọn khác trong code (nhánh 3DGS gốc dùng làm baseline so sánh ở phần đầu, và một vài cơ chế được thiết kế nhưng hiện đang tắt theo mặc định) không được đào sâu thêm ở đây vì không ảnh hưởng tới hành vi mặc định của pipeline.

Bảng so sánh 3DGS gốc vs. SADGS (chỉ những khác biệt ĐANG CHẠY)

Tiêu chí	3DGS gốc (baseline)	SADGS (mặc định)
Tín hiệu densify chính Split dị hướng theo trục Tích lũy đa view trước quyết định	gradient vị trí không có (luôn $N=2$, mọi trục /1.6) không (đọc gradient tại 1 lần cuối)	gradient OR η tần số đa tỉ lệ có, $k_{\text{axis}} = \lceil \sqrt{\max(\eta, 1)} \rceil$ riêng từng trục có ($\text{high_ratio}/\text{low_ratio} > 0.8$ qua nhiều view)
Vị trí Gaussian con khi split	ngẫu nhiên $\mathcal{N}(0, \text{diag}(s)^2)$	lưới tắt định, cách đều $\sqrt{12}\sigma$

*Compact box, bộ lọc low-pass 2D và optimizer hybrid là các cơ chế **chạy giống nhau** ở cả hai nhánh (xem slide "kế thừa vs mới") — không đưa vào bảng so sánh vì không phải điểm khác biệt.*

Ưu điểm cốt lõi của SADGS: hội tụ đúng chỗ, không phải "ít Gaussian hơn"

- **Đừng diễn giải sai:** không có bằng chứng trong repo cho thấy SADGS tạo ra **ít Gaussian hơn** so với 3DGS gốc ở cùng mức chất lượng — cơ chế duy nhất có khả năng chủ động giảm mạnh N ở giai đoạn cuối không nằm trong phạm vi những gì đang chạy mặc định (xem slide "đang chạy mặc định").
- **Lợi ích thực sự, có căn cứ trực tiếp từ công thức đã trình bày:**
 - Phân bổ ngân sách split **đúng trục, đúng mức độ** (k_{axis} tính riêng từng trục, tăng theo bậc thang tương xứng với mức độ vi phạm) thay vì luôn chia đôi đều mọi trục như cơ chế gốc.
 - Quyết định dựa trên **đa số view đồng thuận** ($> 80\%$), giảm hẳn nguy cơ phản ứng theo nhiễu của một góc chụp đơn lẻ bất thường.
 - Tín hiệu η đo trực tiếp **tỉ lệ kích thước Gaussian / chi tiết ảnh**, bổ sung cho gradient (vốn chỉ phản ánh trạng thái hội tụ hiện tại, và có thể bị triệt tiêu nội-ảnh giữa các pixel như đã học).
- **Còn bỏ ngỏ, cần nói rõ khi trình bày kết quả:** repo không có số đo PSNR/SSIM/LPIPS hay wall-clock nào so sánh trực tiếp hai nhánh trên cùng scene/checkpoint để **định lượng** các lợi ích lý thuyết vừa nêu.

Tổng kết: từ 3DGS gốc tới SADGS

- 1 **Biểu diễn & render:** Gaussian 3D tường minh, EWA splatting, alpha blending, rasterizer theo tile — nền tảng chung, giữ nguyên giữa hai nhánh.
- 2 **Gradient:** tích lũy đúng cách (chuẩn trước, cộng sau) tránh triệt tiêu giữa các view; vấn đề triệt tiêu thật nằm giữa các pixel trong cùng một ảnh, được xử lý bằng một kênh gradient trị tuyệt đối riêng.
- 3 **ADC gốc:** clone/split/prune dựa thuần trên gradient — khung sườn chung mà SADGS kế thừa nguyên vẹn.
- 4 **SADGS:** bổ sung tín hiệu η (tỉ số kích thước Gaussian / bước sóng texture cục bộ), tích lũy nhất quán qua nhiều view, split dị hướng theo đúng trục vi phạm — kết hợp OR với gradient chứ không thay thế, và đây là phần **đang chạy mặc định**.
- 5 **Trung thực về hiệu năng:** không có số đo wall-clock/chất lượng nào trong repo để khẳng định "nhanh hơn" hay "ít Gaussian hơn" một cách định lượng — bài giảng này trình bày đúng những gì công thức và code thể hiện, không suy diễn thêm ngoài đó.

Câu hỏi & thảo luận

Toàn bộ công thức trong bài giảng này được đối chiếu trực tiếp với:
`train.py`, `scene/gaussian_model.py`, `utils/freq_utils.py`, `utils/loss_utils.py`,
`arguments/__init__.py`, `cuda_rasterizer/{forward.cu, auxiliary.h}` và `DOCS/BOOK/`.